Software Composition Analysis in the C/C++ Ecosystem: A Survey of Library and Version Identification Methods

Rabie Alawad
IT Security Seminar, Rheinische Friedrich-Wilhelms-Universität Bonn
Bonn, Germany
s6raalaw@uni-bonn.de

Abstract

Software Composition Analysis (SCA) identifies third-party libraries (TPLs) embedded in a software product, a prerequisite for tracking the licensing and vulnerability debt that reused code introduces. Recent supply-chain incidents have made this debt a first-order security concern. Unlike ecosystems with a central package manager, the C/C++ ecosystem provides no such guarantees,libraries are copied, vendored, statically linked, or transformed by the compiler tool-chain. This paper surveys the methods proposed to identify a library and its version inside a C/C++ source tree or stripped binary. The literature is classified along two axes, the analysis target (source vs. binary) and the matching technique (signature-, feature-, structural-, or learning-based), and representative tools are discussed in each category. As its main contribution, the paper consolidates the reported accuracy of these tools into a single comparison table that records the evaluation scope of every figure, making explicit why the numbers are not directly comparable across papers. It closes by examining the gaps that still hold the field back, namely compiler variation, partial and modified reuse, version sensitivity, database quality, cross-language interoperability, and the absence of ground-truth benchmarks, and outlines future directions.

Acronyms

BCSD Binary Code Similarity Detection
CFG Control-Flow Graph
FCG Function Call Graph
FFI Foreign Function Interface
MDL Minimum Description Length
NVD National Vulnerability Database
SBOM Software Bill of Materials
SCA Software Composition Analysis
TPL Third-Party Library

1 Introduction

Modern software is rarely written from scratch; even small applications embed dozens of third-party libraries (TPLs). The benefit is shorter development cycles and access to battle-tested code. The cost is that every imported component drags in its own bugs, license obligations, and security vulnerabilities. Recent supply-chain incidents such as the xz-utils/liblzma backdoor (CVE-2024-3094), the event-stream compromise, and Log4Shell have made the management of this debt a first-order concern. The discipline that tries to keep it visible is called Software Composition Analysis (SCA).

In ecosystems with a strong package manager (Java with Maven, JavaScript with npm, Python with pip), SCA is largely a metadata problem, each project declares its dependencies in a manifest, the SCA tool consumes that manifest, looks up the named components in a vulnerability database, and produces a Software Bill of Materials (SBOM). The C/C++ ecosystem offers no such convenience. Tang et al. [15], in a study of 24,000 GitHub repositories, document 21 different package management tools in active use, none of them universally adopted. Over 70% of dependencies are introduced unintentionally in build scripts rather than through a formal install step, and only 1.8% of repositories use an application-level package manager such as Conan or Vcpkg [15]. Developers vendor source files into their own repositories, copy individual functions, statically link prebuilt archives, or pull libraries through ad-hoc CMake or git-submodule glue. Once compiled, all of this is gone. The resulting binary is a single, often stripped, executable in which library and application code are indistinguishable at the byte level.

Identifying which libraries are present in a C/C++ artifact, and ideally which versions of those libraries, is therefore an active research problem with two sub-problems. Library detection decides whether a TPL is reused. Version identification names the release, since vulnerabilities are tied to specific version ranges. The proposed methods span byte-pattern matching deployed in commercial reverse-engineering tools since the 1990s [6], and recent deep-learning approaches that embed binary functions into vector spaces [2, 20].

The remainder of this paper is structured as follows. Section 2 fixes the terminology and explains why the C/C++ setting is hard. Section 3 states how the surveyed tools were selected and how their reported numbers should be read. Section 4 reviews source-level methods, and Section 5 examines binary-level methods, partitioned into signature-, feature-, structural-, and learning-based families. Section 6 turns to version identification, the hardest of the two sub-problems. Section 7 reports published empirical comparisons and consolidates the headline figures in a single table. Section 9 discusses cross-cutting research gaps, and Section 10 concludes.

2 Background and Terminology

2.1 What “reuse” means in C/C++

Tang et al. [15] analyze the C/C++ dependency lifecycle and find that projects incorporate third-party libraries through a combination of package-management tools and source-code cloning. Dependencies can be fetched via system-level package managers (for example APT on Debian), via application-level managers such as Conan or Vcpkg, or by physically copying library source files into the host repository, a practice also known as vendoring or clone-based reuse. In their study of 24,000 GitHub repositories, code clone alone accounts for 15.77% of all dependencies, and one-third of all repositories use it [15]. The authors observe that this heavy reliance on copy-based reuse is a direct consequence of the C/C++

ecosystem lacking a canonical, universally adopted package manager, there is no standard mechanism to discourage developers from simply pasting library code into their own tree.

Two further notions matter once detection is attempted. Modified reuse describes the case where a library is reused but its source has been altered, for example patched for security, trimmed for size, or adapted for a different platform. Woo et al. [17] report that 95% of components they detected had been modified relative to upstream, which makes exact-match approaches inadequate. Nested reuse refers to the situation where a library L_a itself depends on L_b. If both end up in the target, naive matchers may attribute L_b-style features to L_a or vice versa, producing both false positives and false negatives. Hai et al. [5] show that 41.35% of libraries in a 29,000-repository database lack reliable function birth-time information, breaking the simplest filtering strategy for nested reuse.

2.2 The compilation gap

The central difficulty of binary-level SCA is that the compiler is not a structure-preserving transformation. A single C function may translate to dozens of different instruction sequences depending on the architecture (x86-64, ARM, RISC-V), the compiler (GCC, Clang, MSVC), the optimization level (`-O0` through `-O3`), and a long list of code-generation flags. Inlining, function cloning, vectorization, and link-time optimization further blur function boundaries. Distributors typically run `strip` on release binaries, removing the symbol table that would have made identification trivial. Every binary-level SCA tool therefore faces the same design choice, pick a feature representation stable enough across compilation choices to still match a known library, yet specific enough not to match unrelated code. The history of the field can be read as progressively richer answers to that question.

2.3 The general SCA pipeline and how results are measured

Nearly all SCA tools follow the same two-phase structure. In the offline phase, a large collection of known open-source libraries is processed in advance and stored as a feature database, where each library version is represented as hashed function bodies, string literals, or learned embeddings. In the online phase, the target software goes through the same feature extraction and is compared against the database. If the similarity to a known library exceeds a threshold, the tool reports that library as present (library detection); if it can additionally name the specific release, it performs version identification. This second step is what ultimately matters for vulnerability management, since the NVD lists affected version ranges, not just library names.

Results are measured with precision (the fraction of reported detections that are correct) and recall (the fraction of libraries actually present that the tool finds). Because each paper constructs its own evaluation dataset and no community benchmark exists [11], numbers are not directly comparable across papers. How this survey handles that comparability problem is set out in Section 3, and its consequences are revisited in Section 7.

3 Methodology

This survey is organized around two questions: which tools represent the state of the art in C/C++ library and version identification, and how their reported effectiveness should be read. This section states how the surveyed work was selected and how its quantitative claims are compared.

3.1 Literature selection

This survey covers techniques whose primary goal is to identify a TPL, or its version, inside a C/C++ source tree or compiled binary. Tools were selected along three criteria. First, the method targets the C/C++ ecosystem, or its technique was directly adopted by C/C++ tools, for example the Android tool ATVHunter [23], whose two-stage coarse-to-fine scheme reappears in several C/C++ systems. Second, the work was published at a peer-reviewed venue between 2017 and 2025, a span that captures the shift from byte-pattern matching to learned embeddings; the one exception is FLIRT [6], retained as the pre-2017 industrial baseline that later academic work repeatedly measures itself against. Third, the tool is representative of a distinct technical idea rather than an incremental variant, so that each one discussed illustrates a different point in the design space rather than a marginal tuning of a neighbour.

Applying these criteria yields the tools summarized in Table 1, organized along the two axes used for the rest of the paper. The first axis is the *analysis target*: source-level methods (Section 4) assume access to the codebase, whereas binary-level methods (Section 5) operate on compiled, often stripped, artifacts. The second axis is the *matching technique*: byte-pattern signatures, string and metadata features, structural and semantic features, and learning-based embeddings, in roughly chronological but cumulative order.

3.2 How the reported figures are compared

Throughout this survey, effectiveness is reported with precision and recall, quoted from each tool's own paper. These figures are not directly comparable across tools, because every paper builds its own evaluation dataset and no community benchmark exists [11]: the targets differ (source trees, statically linked binaries, cross-language binaries), the task differs (whole-library detection, partial-reuse detection, version identification), and a few tools report only accuracy or only F1 rather than a precision and recall pair. The numbers are nonetheless retained because each is the headline result its authors stand behind, and together they indicate the order of magnitude and the characteristic precision-recall trade-off of each method family.

To keep this limitation visible rather than hidden, every figure in Table 1 is annotated with the evaluation it comes from. Two kinds of controlled comparison are distinguished from self-reported numbers. First, three papers re-evaluate earlier tools on a shared dataset: OSSFP [18] re-runs CENTRIS on a matched test scope, LibAM [9] re-runs LibDX, B2SFinder, and LibDB on the public Dataset_ISRD, and BinCoFer [24] re-runs ModX, LibAM, B2SFinder, and BinaryAI on its ArchLinux set. Rows drawn from such re-evaluations are marked *re-eval* in the table. Second, the same tool can score very differently on two datasets: LibAM reports 100/99.3 on Dataset_ISRD but 38.4/93.6 on BinCoFer's set, which is the clearest single illustration of why a raw cross-paper ranking would

be misleading. The empirical comparisons in Section 7 rely only on these controlled re-evaluations, not on the self-reported columns.

4 Source-Level Methods

Source-level SCA is relevant when the consumer of the library is the developer themselves, when the auditor has access to the codebase during a license-compliance review, or when binary tools rely on a clean source-side database internally.

4.1 Manifest-driven detection: CCScanner

The simplest source-level method parses package management configuration files and build-system files, resolving them against package metadata. CCScanner [15] parses "quasi-SBOM" files, namely configuration files for package management tools and build systems that effectively describe required dependencies, across 21 different tools including Deb, Conan, Vcpkg, CMake, Autoconf, Bazel, Meson, MSBuild, and git submodules. The term quasi-SBOM reflects the fact that, while these files were not designed as formal dependency manifests, they contain sufficient structured information to extract dependency relationships from, provided one understands the syntax of each respective tool. CCScanner additionally integrates CENTRIS [17] as a code-clone backend to recover dependencies introduced by copy-based reuse. On a 15-project benchmark of 589 manually labeled dependencies, CCScanner achieves 86% precision and 80.1% recall [15]. The same study evaluates CCScanner and the two established SBOM scanners, OWASP Dependency-Check and Sonatype, on this one shared ground truth, so the figures are directly comparable. The paper reports two recall measures. Against the full ground truth, OWASP reaches only 27.2% and Sonatype 0%, far below CCScanner's 80.1%, because each scanner supports only a subset of the package-management formats that CCScanner parses. Restricted to the formats each tool does support, the gap largely disappears, with OWASP recalling 83.3% against CCScanner's 80.8% and at a higher precision of 93.9%. The advantage of CCScanner is therefore breadth of format coverage rather than more accurate matching, while Sonatype, which reads only Conan manifests, detects nothing in the Conan-free projects of this benchmark [15].

The manifest-parsing approach derives its strength from precision. When a `CMakeLists.txt` explicitly declares `find_package( OpenSSL 3.0 REQUIRED)`, no fingerprinting or heuristic matching is required, and the dependency is explicit and unambiguous. Its fundamental weakness, however, lies in recovering copy-based reuse. When library code is directly copied into a repository, no manifest entry is created, making such dependencies invisible to static file parsing alone.

4.2 Code-clone detection at function granularity

To cover copy-based reuse, the second source-level family applies code-clone detection at function granularity. CENTRIS [17] is the de facto baseline. It extracts each C/C++ function, normalizes its body (whitespace, comments, lexical noise), and computes a TLSH-style hash. The TPL repository stores hashes for every function of every version of every candidate library; the target is matched by hash matching at function granularity. The key insight is a code-segmentation step that determines, for each candidate library, which functions are "native" by checking when each first appeared in version history, discarding "borrowed" code from the comparison. On 10,241 GitHub C/C++ repositories spanning 80 billion lines of code, the authors report 91% precision and 94% recall.

Three more recent tools refine this baseline in different directions. OSSFP [18] observes that the dominant source of false positives is shared utility code, which the authors call support functions and common functions. OSSFP categorizes functions into four classes (clone, support, common, core) and keeps only core functions in the fingerprint index, eliminating the threshold-based matching that CENTRIS requires and that drives false negatives on partially reused libraries. Built on a fingerprint database of 23,427 C/C++ repositories with 585,683 versions and 90 billion lines of code, OSSFP reaches 90.84% recall and 90.34% precision on its 896-TPL ground truth, processing each project in 0.12 seconds, 22 times faster than CENTRIS. Its reported 35.31-percentage-point recall advantage comes from re-evaluating CENTRIS on this benchmark, where CENTRIS generalizes far worse than the 94% recall it reports on its own dataset. On the matched 657-case test scope, OSSFP recalls 92.38% against CENTRIS' 57.07% [18].

OSSDetector [5] addresses three weaknesses simultaneously including brittleness against modified code, unreliable nested-TPL filtering, and lack of version identification. Instead of hashing whole functions, it uses a sliding-window scheme where a window slides byte-by-byte across each function, and a partition is created whenever `Adler-32(window) mod n = n-1`, then applies ssdeep fuzzy hashing to each block. This makes the resulting signature tolerant to localized modifications. For nested filtering, the tool replaces the single-feature birth-time test with a Nested TPL Function Filtering Algorithm that combines six features including associations between TPLs, code comments, library contributor information, function birth times, library names, and file paths. The authors report 85.52% library-level precision and 79.82% recall on a 1,199-TPL benchmark, with 84.27% precision at the library-version level.

TPLite [7] questions the function-birth-time assumption that CENTRIS depends on. Birth times can be missing or wrong, for example when a library migrates to GitHub late and earlier versioning is lost, and this directly limits the precision of nested-reuse filtering. TPLite combines hierarchical path information and metadata (header files, SBOM files, licenses) to determine the original TPL of each function, then applies graph-based dependency recall to remove invalid dependencies. On a 1,035-TPL, 128-binary benchmark, TPLite outperforms both CENTRIS and the commercial Black Duck Binary Analyzer at the binary-level SCA task.

In short, source-level detection is accurate when the auditor has the code, but its recall is bounded by how completely the feature database covers the libraries actually in use.

5 Binary-Level Methods

The bulk of the C/C++ SCA literature targets the binary case. Methods are organized in four groups by feature type, namely byte-pattern signatures, string and metadata features, structural and semantic features, and learning-based embeddings, roughly chronological but cumulative.

5.1 Byte-pattern signatures: FLIRT and BinShape

To address library function identification in binaries, security researchers have developed techniques to automatically recognize library functions. A widely-used approach is IDA FLIRT (Fast Library Identification and Recognition Technology), which applies signature matching to patterns generated from the first invariant byte sequence of each function [4, 12]. FLIRT builds each signature from the first 32 bytes of a function, wildcarding the bytes that vary when the library is loaded, such as relocations [12]. To reduce the collisions that arise when several functions share the same leading bytes, FLIRT additionally stores a CRC-16 computed over the bytes following the pattern together with the total function length [4]. Signatures are organized in a tree keyed on the non-wildcard byte sequences, so a target function can be matched efficiently.

Despite its efficiency, FLIRT's reliance on raw byte patterns makes it brittle. Shirani et al. [12] observe that it suffers from signature collisions and frequently requires a fresh signature for each new version, because different compilers and build options alter the byte-level patterns. Consequently, the signature database must contain entries for the specific library version, platform, and compiler configuration used to build the target, which in practice limits FLIRT's coverage to standard runtime libraries and a small set of widely deployed packages.

To overcome this brittleness, BinShape [12] generalizes the signature paradigm beyond raw bytes. It derives a robust per-function signature, the "shape" of the function, from a diverse set of features spanning control-flow-graph metrics, instruction-level characteristics, statistical features, and function-call graphs. Evaluated on C/C++ binaries compiled with GCC and Visual Studio at optimization levels `-O0` to `-O3`, BinShape attains an average F1 of 0.89, against 0.81 for FLIRT on the same binaries. The gap widens under stress. When obfuscation techniques are applied, FLIRT degrades to an F1 of 0.76 while BinShape holds at 0.86, and across changes in compiler family and optimization level BinShape's precision and recall stay consistently higher (for example, 1.00/1.00 versus FLIRT's 0.91/0.74 on `libpng` under a changed optimization level). These results show that signature-based identification can tolerate compilation variability when the underlying features are chosen to be resilient to it.

5.2 String and metadata features

A second family observes that, even after stripping, binaries retain large amounts of read-only data (format strings, error messages, version strings, embedded configuration) that are robust under recompilation and library-specific.

OSSPolice [3] builds a hierarchical TPL repository keyed on string literals, exported function names, and fuzzy code signatures, organized to mirror the package structure of the upstream open-source project. A binary is matched by walking the hierarchy from coarse (does this binary share strings with any project?) to fine (which class or module of that project?). B2SFinder [22] refines this for binary-to-source TPL detection in commercial-off-the-shelf software. It extracts seven feature classes including string literals, switch/case constants, branch constants from if/else statements, exported function names, and three array-valued features (string, integer, enum arrays). The tool combines per-feature scores with two importance-weighting functions accounting for both specificity in the candidate library and frequency across the corpus. LibDX [13] takes a deliberately minimalist line. It reads only the `.rodata` (read-only data) section, where string constants such as messages and format strings live, and finds that these strings alone, once duplicates shared across libraries are removed, are often enough to tell libraries apart, optionally aided by fuzzy filename hints. LibDX groups the extracted strings into "logic feature blocks" defined by their position in the binary and matches whole blocks rather than individual strings, relying on string data rather than any analysis of the disassembled instructions.

5.3 Structural and semantic features

To go beyond strings, the third family lifts the analysis to the function level and uses control-flow or call-graph structure. BB-Detector [16] embeds each binary function into a feature vector combining syntactic features (constants in assignments, constant parameters in function calls) with structural features (in-degree and out-degree on the call graph). Matching proceeds in two stages. First a vector-similarity search identifies candidate "anchor" functions and the libraries they belong to, then a finer comparison confirms the match.

ModX [21] targets the partial-reuse problem directly. Real-world reuse rarely imports an entire library; developers cherry-pick the few functions they need, so whole-library matchers suffer on recall. ModX decomposes both the target binary and the candidate library into fine-grained modules by applying community-detection algorithms over the call graph, and then matches modules using syntactic and semantic features. Because reuse is reported when only a subset of modules matches, ModX recovers partially imported libraries that whole-library matchers miss.

LibAM [9] addresses the same partial-reuse problem with a different strategy that avoids fixed modular boundaries. It identifies "areas" on the function call graph, connected subgraphs whose constituent functions are individually similar to functions in the candidate library, and matches at the area level rather than per function or per whole library. Because areas are grown from the call graph rather than produced by a clustering algorithm, they can overlap; an Anchor Alignment algorithm resolves the resulting ambiguity. LibAM also reports the exact reused area, a substantial usability gain, since the auditor learns not only that a library is present but precisely which of its functions were reused, enabling more targeted vulnerability triage.

5.4 Learning-based methods

The most recent family treats binary-level SCA as a downstream task of binary code similarity detection (BCSD) and exploits learned function embeddings. The conceptual frame is set by BCSD models that map a binary function to a fixed-length vector such that semantically similar functions are close in the vector space. Asm2Vec [2] adapts the PV-DM word-embedding model to assembly. Gemini [20] uses graph neural networks over attributed control-flow graphs for cross-architecture matching. Comparative evaluations show no single embedding dominates across architectures and optimization

levels, but all of them substantially outperform pre-deep-learning function similarity tools [10].

LibDB [14] was among the first to fold a learned function embedding directly into a TPL detection pipeline. In addition to the coarse string and exported-function features used by earlier tools, LibDB embeds every function in the binary into a low-dimensional vector with a trained network and compares them against the embeddings of all candidate library functions. A function-call-graph-based comparison aggregates per-function evidence into a library-level decision, and, notably, LibDB additionally supports version identification of the detected library, a capability earlier binary detectors did not provide.

BinaryAI [8] extends this to a binary-to-source setting. Source functions from a corpus of 12,013 C/C++ TPLs are embedded offline into a vector database. At analysis time, the tool decompiles the target binary, embeds each binary function with the same model, and retrieves the top-k similar source functions. A second "locality-driven matching" phase selects the source function most consistent with the target's link-time locality and call-graph neighborhood. On a manually labeled 150-binary, 1,045-TPL benchmark, BinaryAI reaches 85.84% precision and 64.98% recall, outperforming all four baselines evaluated in the same study, the academic tools OSSPolice [3] and B2SFinder [22] and two commercial products. The comparison is controlled. Every tool ran on the same benchmark with its detection database built from the same 12,013-project TPL corpus, so the figures are directly comparable. The margin is wide. The strongest academic baseline, B2SFinder, reaches only 31.78% precision and 54.93% recall, and the stronger of the two commercial baselines trails BinaryAI at 73.36% precision and 59.81% recall.

BinCoFer [24] targets the binary-to-binary setting and addresses two problems earlier function-level tools largely ignored: feature-repository redundancy and the ambiguity of single-function similarity scores. It applies a three-stage purification process to identify each library's "core" functions and discards the rest, then aggregates similarities across all core functions of a candidate library using a weighted voting scheme. The authors report a precision improvement of more than 20 percentage points and an F1 gain of 7 points over ModX, B2SFinder, LibAM, and BinaryAI on an ArchLinux-derived dataset.

DeeperBin [19] is the most recent entry and confronts a problem the earlier tools largely overlooked, namely cross-language binaries. In modern polyglot projects, C/C++ TPLs are routinely invoked from Python, Rust, or other host languages through Foreign Function Interface (FFI) bindings, and the FFI glue itself shows up as features that obscure the underlying TPL. DeeperBin uses a feature database covering 74,647 TPL versions, an explicit FFI-binding filter that removes glue functions before matching, automated extraction of version-string regular expressions for 31,855 TPL versions, and Minimum-Description-Length (MDL) selection of distinctive version features. On 418 cross-language binaries it reports 81.2% precision and 84.6% recall for detection and 70.3% precision for version identification, with the FFI filter and MDL features together contributing 17.1 and 18.8 percentage points to precision and recall respectively in an ablation.

6 Version Identification

Detecting that a library is present is useful, but identifying that the binary uses, say, OpenSSL 1.0.1f rather than 1.0.1g is what enables vulnerability triage. Version identification is fundamentally harder than detection because the features that survive compilation most reliably, namely exported names, format strings, and control-flow shapes, are also the ones least likely to change between adjacent releases.

Older approaches handled version identification opportunistically. OSSPolice [3] scans for version-stamp strings and otherwise approximates the version by selecting the features unique to a single version of a library. Ban et al. [1] show that adjacent versions often differ in only a handful of functions, so this approach cannot distinguish them with confidence.

B2SMatcher [1] was the first tool to make version identification its primary objective. The authors systematically catalogue five classes of version-sensitive code features jointly trackable in source and binary, comprising program-level features (string literals, exported function names) and function-level features (constants in assignments, constant parameters in function calls, call-graph in/out-degrees). They compute a discrimination score for each feature class and retain those exceeding an empirical threshold. Detection runs in two stages. Program-level features select candidate versions and reuse types (single-version or multi-version), and function-level features pick the precise version. To make function comparison tractable across thousands of versions, the tool uses machine learning to predict which source files are actually compiled into the binary and applies function abstraction and normalization to deduplicate functions unchanged between versions. On 6,351 candidate OSS versions and 585 binaries, B2SMatcher reaches 89.2% precision, which is 8 and 7 percentage points above OSSPolice and a commercial baseline.

OSSDetector [5] extends function-level matching to the library-version level rather than stopping at library detection. Building on its fuzzy function hashing and nested-TPL filtering, it reports 84.27% precision at the library-version level, the highest reported in the source-level literature.

ATVHunter [23], although developed for Android, demonstrated a related coarse-to-fine two-stage scheme that recurs in C/C++ tools, where program-level coarse-grained features identify the library and function-level fine-grained features pinpoint the version.

DeeperBin [19] contributes the most explicit treatment of version-distinguishing features for the binary case. It extracts version-string regular expressions automatically for tens of thousands of TPL versions and uses the MDL principle to select features distinctive across versions, since features appearing in many versions carry little information about which version is actually present. LibDB [14] also supports version identification within its embedding-based binary pipeline, a capability most earlier binary detectors lacked. Despite these advances, version identification remains the most fragile part of the pipeline. Ning et al. [11] find that the SCA tools they evaluate all lose substantial precision when moving from detecting a library to naming its version, and adjacent patch versions are particularly hard because differences are often confined to a handful of lines.

Version identification is therefore consistently weaker than detection, because the features that survive compilation are also the ones least likely to change between adjacent releases.

7 Empirical Comparisons

Two recent empirical studies provide the most reliable cross-tool comparisons the field has produced. Ning et al. [11] evaluate three commercial SCA platforms (Black Duck, Scantist, Cybellum) and one academic tool (BinaryAI) on 16,713 C/C++ binaries spanning common libraries, combined libraries, manually obfuscated builds, and reverse-engineered automotive firmware. Their headline numbers are sobering. Precision and recall for general libraries are above 70%, but drop below 45% when the targets are domain-specific automotive firmware. The study also documents large performance differences. Scantist scans a typical file in under a second, while Cybellum takes 9 to 34 seconds because it performs much more comprehensive per-binary analysis. The tools also differ markedly in robustness to obfuscation, with the detectors that rely on more robust semantic features holding up better than those depending on raw byte matching.

Jiang et al. [7], in the TPLite paper, contribute a complementary cross-source-tool evaluation. On a curated benchmark of 1,035 TPLs and 128 binaries, TPLite outperforms the commercial Black Duck Binary Analyzer (precision 75.90% vs. 72.46%, recall 64.17% vs. 58.55%). Database completeness is also a determining factor for any code-clone-based tool. Tang et al. [15] find that CENTRIS' coverage of popular libraries can fall below 50% when its database lacks the relevant repositories. A consistent finding across these studies is that no current tool dominates. Industrial tools have richer engineering, but academic tools that focus on a single technical idea match or exceed them on the specific aspect they target, so practical SCA will need to combine methods rather than rely on any single tool.

8 Interpreting the Reported Results Across Studies

Table 1 consolidates the headline precision and recall figures discussed throughout this survey, grouped by method family. As emphasized in Section 3, the figures are not directly comparable: each cell records the evaluation it comes from, and only the re-evaluation rows constitute controlled cross-tool comparisons. The table should therefore be read as a map of the design space rather than a ranking. At first glance many tools look strong, several reporting both precision and recall above 80%, but those numbers originate from different datasets, tasks, and occasionally different metrics, so the appearance of a common scale is misleading.

The clearest structural pattern is the gap between source-level and binary-level approaches. Source-level tools such as CENTRIS and OSSFP tend to report balanced precision and recall above 90%, because they operate on code in which structural information is preserved and clone detection is comparatively reliable. Binary-level tools confront the compilation gap of Section 2 (optimization, symbol stripping, and platform variation) and their reported effectiveness spreads much more widely, depending on the feature representation and the evaluation setting.

The re-evaluation rows expose how much of the self-reported optimism is an artifact of in-distribution testing. A tool measured on an independent benchmark almost always scores below its original paper. CENTRIS reports 94% recall on its own dataset, yet OSSFP's re-evaluation on a matched 657-case scope measures only 57.07% recall [18]. The effect is even starker for LibAM, which reaches 100% precision and 99.3% recall on Dataset_ISRD but drops to 38.4% precision when BinCoFer re-runs it on an ArchLinux-derived set [24]. Such swings show that dataset characteristics, not algorithmic differences alone, dominate the reported figures, and that generalization across environments is still an unsolved problem.

A further pattern is the deliberate precision–recall trade-off among binary-level methods. Coverage-oriented tools such as LibDB attain very high recall at low precision (55.3% / 93.6%), surfacing many candidate libraries at the cost of false positives. Precision-oriented tools such as BinaryAI (85.84% / 64.98%) and BinCoFer (89.3% / 64.9%) suppress false detections while conceding recall. No single operating point dominates, which is itself an argument that production deployments will need to combine complementary techniques rather than rely on one matcher.

Version identification is the hardest task represented in the table. Even the strongest entries, B2SMatcher at 89.2% and OSSDetector at 84.27% version-level precision, fall below the same tools' library-detection scores, and several studies report no version-level recall at all, which makes like-for-like comparison impossible. This reflects the underlying difficulty noted in Section 6: adjacent releases often differ in only a handful of functions, precisely the features most likely to survive compilation unchanged.

Taken together, Table 1 traces the evolution of the field from byte-pattern matching toward structural, semantic, and learning-based methods, while simultaneously exposing its central methodological weakness. The absence of shared, version-labeled benchmarks means progress cannot yet be measured on a common scale, and closing that gap is as important to the field's maturity as any new detection technique.

9 Research Gaps and Future Directions

Looking across the literature surveyed, several recurring difficulties stand out, and 2024–2025 work hints at where the field is heading.

9.1 Compiler-induced variation

Every binary-level method must reckon with the fact that the same source produces different binaries under different compilers and optimization levels. Empirical evaluations consistently show double-digit accuracy drops when the analysis target was compiled with settings differing from those used to build the feature repository [11]. Learning-based methods generalize better than byte-pattern signatures, but no method is fully invariant; recent BCSD evaluations observe a measurable gap between same-compiler/same-optimization pairs and cross-compiler/cross-optimization pairs even for the best embeddings [10].

Table 1: Reported precision and recall of each surveyed tool, grouped by method family, with the evaluation each figure comes from. Figures are not directly comparable, since each tool is measured on a different dataset, task, and sometimes a different metric (see Section 3). Where a tool has been measured under more than one setting, each setting appears on its own row. Rows marked *(re-eval)* are a controlled re-evaluation of an earlier tool by a later paper; rows marked *(baseline)* are the numbers a study reports for the tools it compares against; rows marked *(ver.-level)* report version-identification accuracy rather than library detection, where LibDB and the OSSPolice baseline use version precision (VP), the exact-version rate among detected libraries. The relevant study is identified in the scope column.

Family	Tool	Prec.	Rec.	Evaluation scope
Manifest parsing (source)	CCScanner [15]	86%	80.1%	15-project benchmark (own)
	OWASP *(baseline)*	93.9%	27.2%	CCScanner's ground truth; full-GT recall (R2 83.3%)
Code-clone, function-level (source)	CENTRIS [17]	91%	94%	own large-scale set
	CENTRIS *(re-eval)*	63.88%	57.07%	657-case shared scope, by OSSFP [18]
	OSSFP [18]	67.59%	92.38%	657-case shared scope, by OSSFP [18]
	OSSFP [18]	90.34%	90.84%	896-TPL ground truth (own)
	OSSDetector [5]	85.52%	79.82%	1,199-TPL benchmark (own); F1 82.57%
	TPLite [7]	75.90%	64.17%	1,035-TPL / 128-binary (own)
	Black Duck *(baseline)*	72.46%	58.55%	in TPLite's binary benchmark
Byte-pattern signature (binary)	FLIRT [6]	—	—	F1 81% in BinShape's evaluation (baseline); no standalone benchmark
	BinShape [12]	—	—	F1 89% on the same benchmark; GCC/MSVC `-O0`–`-O3` (own)
String & metadata (binary)	OSSPolice [3]	82%	87%	Android-app task; no P/R reported
	B2SFinder [22]	69.2%	64.4%	re-eval on Dataset_ISRD, by LibAM [9]
	LibDX [13]	92%	80.9%	re-eval on Dataset_ISRD, by LibAM [9]
Structural & semantic (binary)	BBDetector [16]	—	—	reports F1 only, 77% non-obf./71% obf. (own)
	ModX [21]	84.3%	61.7%	real-world programs (own)
	ModX *(self-crafted)*	90.1%	78.2%	self-crafted programs (own)
	LibAM [9]	100%	99.3%	Dataset_ISRD (own)
	LibAM *(re-eval)*	38.4%	93.6%	40-binary set, by BinCoFer [24]
Learned embeddings (binary)	LibDB [14]	55.3%	93.6%	TPLBinary dataset (own)
	BinaryAI [8]	85.84%	64.98%	150-binary, 1,045-TPL (own)
	B2SFinder *(baseline)*	31.78%	54.93%	in BinaryAI's benchmark
	Black Duck *(baseline)*	73.36%	59.81%	in BinaryAI's benchmark
	BinCoFer [24]	89.3%	64.9%	40-binary ArchLinux set (own)
	DeeperBin [19]	81.2%	84.6%	418 cross-language binaries (own)
Version identification	B2SMatcher [1]	89.2%	—	585 binaries (own)
	OSSDetector *(ver.-level)*	84.27%	78.65%	1,199-TPL benchmark (own) [5]
	DeeperBin *(ver.-level)*	70.3%	—	418 cross-language binaries (own) [19]
	LibDB *(ver.-level)*	60.4%	—	TPLBinary dataset; version precision (VP) [14]
	OSSPolice *(ver., baseline)*	40.2%	—	VP, as LibDB's baseline [14]
	OSSPolice [3]	82%	Version Match (VM): 55/67	
	ATVHunter [23]	90.55%	87.16%	Android version-level, F1 88.82% (out of scope)

9.2 Partial reuse and modified reuse

As several recent tools emphasize [5, 9, 21, 24], real software rarely imports a complete library. Developers cherry-pick the few functions they need, so whole-library matchers either miss these cases or wash out the signal in unmatched-function noise. Modular and area-based matchers recover much of the lost recall, but at a substantial runtime cost; on BinCoFer's benchmark, for example, ModX and LibAM take on the order of hundreds to thousands of seconds per binary against BinCoFer's single-digit seconds [24]. Modified reuse compounds the problem. Woo et al. report that 95% of the components detected in CENTRIS' study had been altered relative to upstream [17], so exact-match approaches are inadequate, and OSSDetector's sliding-window fuzzy hashing is an explicit acknowledgement of exactly this [5].

9.3 Feature-repository quality

The effectiveness of an SCA tool is largely a function of how good its underlying TPL database is. Tang et al. [15] show that the C/C++ ecosystem has no complete library source: even a database that merges all three common source categories (system libraries, package-manager repositories, and crawled GitHub projects) covers only 77% of observed dependencies, and the best single source reaches just 65.5%. Their study also finds that CENTRIS' coverage of the top-100 popular libraries falls below 50% when the relevant repositories are absent from its database. Inter-library code duplication compounds the problem by injecting non-distinguishing features. The birth-time segmentation of CENTRIS [17], the core-function extraction of OSSFP [18], the three-stage purification of BinCoFer [24], and the six-feature nested filtering of OSSDetector [5] are all responses to the same observation, that a TPL database is only as useful as it is clean. The largest reported database to date covers 29,416 TPLs and 767,405 versions [5], which itself indicates how much progress comes from data engineering rather than algorithmic innovation.

9.4 Cross-language and FFI-bound binaries

DeeperBin [19] highlights a problem almost no earlier work addressed, namely cross-language binaries. In modern polyglot projects, C/C++ TPLs are routinely invoked from host languages such as Python, Rust, Go, or Node.js through Foreign Function Interface (FFI) bindings, and the glue code that implements those bindings appears in the binary as noise relative to the underlying TPL features. In DeeperBin's ablation, the FFI filter together with MDL-based feature selection boosts precision and recall by 17.1% and 18.8% respectively, a substantial part of the tool's effectiveness. This indicates that tools which do not account for FFI glue are penalized on cross-language binaries, a difficulty that earlier evaluations largely overlooked. DeeperBin also begins to address fused binaries, where multiple TPLs are compiled into a single executable with blurred boundaries, but a systematic treatment of those boundaries remains an open problem.

9.5 The ground-truth bottleneck

Evaluating an SCA tool requires a benchmark that pairs binaries with the libraries actually present in them, and constructing such benchmarks at scale is labor-intensive. Most published evaluations rely on tens to a few hundred manually curated binaries rather than thousands, for example 40 in BinCoFer, 128 in TPLite, and 150 in BinaryAI. Tang et al. [15] note explicitly that no public test data exists for C/C++ dependency detection, so each new paper effectively constructs its own ground truth. This limits cross-paper comparison and creates a risk of method-specific overfitting, and it is the deeper reason the figures in Table 1 cannot be read as a direct ranking. The construction of community-maintained, large-scale, version-labeled benchmarks remains an open organizational problem.

9.6 Source-level vs. binary-level convergence

A final observation is that the source- and binary-level branches have begun to converge. CENTRIS feeds binary-to-source tools by serving as a clean source-side database; BinaryAI [8] bridges the two sides explicitly by embedding both binary and source functions into a shared vector space; CCScanner [15] integrates manifest parsing with code-clone detection; OSSDetector [5] links source-level detection to a CVE-aligned vulnerable-TPL database. The empirical evidence [7, 11] argues that this convergence is necessary: no single feature type captures all reuse patterns in C/C++, and any tool aiming for production-grade recall has to combine manifest parsing, source-level cloning, and binary-level matching.

10 Conclusion

Software Composition Analysis in the C/C++ ecosystem is defined by the absence of structure that other ecosystems take for granted. Without a canonical package manager, without preserved metadata in the binary, and with a compiler tool-chain that aggressively rewrites whatever source-level identity a library once had, every method must reconstruct the bill of materials from indirect evidence.

This paper has surveyed the methods developed to do so. Source-level methods rely on either parsing the build system [15] or detecting code clones across normalized function bodies [5, 7, 17, 18], and have largely solved the case where the auditor has the source tree, modulo the database-coverage problem. Binary-level methods divide along their feature representations: byte-pattern signatures [6, 12] are fast but brittle; string- and metadata-based methods [3, 13, 22] are robust but largely version-blind; structural and semantic methods [9, 16, 21] recover partial-reuse recall and call-graph-level interpretability; and learning-based methods [8, 14, 19, 24] close much of the precision gap to commercial offerings while remaining open and reproducible. Version identification, treated as a first-class objective by B2SMatcher [1], OSSDetector [5], and DeeperBin [19], is markedly harder than detection and remains the least-resolved sub-problem.

Two cross-cutting lessons emerge from the survey. First, as the re-evaluation rows of Table 1 show, a tool's effectiveness is inseparable from the dataset it is measured on, and the field's reliance on per-paper ground truth makes published rankings unreliable. Second, the empirical studies [7, 11] agree that no single feature type captures every reuse pattern, so no single tool dominates.

These observations point to three directions. The first is hybrid pipelines that combine manifest parsing, source-level clone detection, and binary-level embeddings, so that each component compensates for the others' blind spots. The second is the explicit handling of cross-language and FFI-bound binaries, where DeeperBin's filtering result suggests substantial accuracy gains remain on the table. The third, and most fundamental, is the construction of community-maintained, large-scale, version-labeled benchmarks, without which the field cannot measure its own progress. Together these suggest that the next generation of C/C++ SCA tools will look less like a single matcher and more like a small ensemble, evaluated against a shared standard the field does not yet have.

References

[1] Gu Ban, Lili Xu, Yang Xiao, Xinhua Li, Zimu Yuan, and Wei Huo. 2021. B2SMatcher: Fine-Grained Version Identification of Open-Source Software in Binary Files. *Cybersecurity* 4, 1 (2021), 21.

[2] Steven H. H. Ding, Benjamin C. M. Fung, and Philippe Charland. 2019. Asm2Vec: Boosting Static Representation Robustness for Binary Clone Search Against

Code Obfuscation and Compiler Optimization. In *Proc. IEEE Symp. on Security and Privacy (S&P)*. 472–489.

[3] Ruian Duan, Ashish Bijlani, Meng Xu, Taesoo Kim, and Wenke Lee. 2017. Identifying Open-Source License Violation and 1-day Security Risk at Large Scale. In *Proc. 2017 ACM SIGSAC Conf. on Computer and Communications Security (CCS)*. 2169–2185.

[4] Chris Eagle. 2011. *The IDA Pro Book: The Unofficial Guide to the World's Most Popular Disassembler*. No Starch Press.

[5] Xun Hai, Jie Zeng, Zhe Fu, Sen Chen, Yi Shi, and Hao Sun. 2025. OSSDetector: Towards a More Accurate Approach for C/C++ Third-Party Library Detection. In *Proc. 24th IEEE Int. Conf. on Trust, Security and Privacy in Computing and Communications (TrustCom)*.

[6] Hex-Rays SA. 2024. FLIRT Technology: In-Depth. https://docs.hex-rays.com/user-guide/signatures/flirt. Accessed: 2026-06-28.

[7] Ling Jiang, Hengchen Yuan, Qian Tang, Sen Nie, Shi Wu, and Yuan Zhang. 2023. Third-Party Library Dependency for Large-Scale SCA in the C/C++ Ecosystem: How Far Are We?. In *Proc. 32nd ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*. 1383–1395.

[8] Ling Jiang, Hengchen Yuan, Jiyou Wu, Junwen Wang, Jianjun Zhang, Yang Liu, and Bin Liu. 2024. BinaryAI: Binary Software Composition Analysis via Intelligent Binary Source Code Matching. In *Proc. 46th IEEE/ACM Int. Conf. on Software Engineering (ICSE)*. Article 100.

[9] Siyuan Li, Yongpan Wang, Chaopeng Dong, Shouguo Yang, Hong Li, Hao Sun, Zhe Lang, and Hongsong Chen. 2024. LibAM: An Area Matching Framework for Detecting Third-Party Libraries in Binaries. *ACM Trans. on Software Engineering and Methodology* 33, 2 (2024), Article 35.

[10] Andrea Marcelli, Mariano Graziano, Xabier Ugarte-Pedrero, Yanick Fratantonio, Mohamad Mansouri, and Davide Balzarotti. 2022. How Machine Learning Is Solving the Binary Function Similarity Problem. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2099–2116. https://www.usenix.org/conference/usenixsecurity22/presentation/marcelli

[11] Yiran Ning, Ying Zhang, Chunyang Ma, Zhi Guo, and Le Yu. 2024. Empirical Study of Software Composition Analysis Tools for C/C++ Binary Programs. *IEEE Access* 12 (2024), 50418–50430.

[12] Paria Shirani, Lingyu Wang, and Mourad Debbabi. 2017. BinShape: Scalable and Robust Binary Library Function Identification Using Function Shape. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA), LNCS 10327*. 301–324.

[13] Wei Tang, Yanlin Wang, Hongyu Zhang, Shi Han, Ping Luo, and Dongmei Zhang. 2020. LibDX: A Cross-Platform and Accurate System to Detect Third-Party Libraries in Binary Code. In *Proc. 27th IEEE Int. Conf. on Software Analysis, Evolution and Reengineering (SANER)*. 104–115.

[14] Wei Tang, Yanlin Wang, Hongyu Zhang, Shi Han, Ping Luo, and Dongmei Zhang. 2022. LibDB: An Effective and Efficient Framework for Detecting Third-Party Libraries in Binaries. In *Proc. 19th Int. Conf. on Mining Software Repositories (MSR)*. 423–434.

[15] Wei Tang, Zhengzi Xu, Chengyue Liu, Jiahui Wu, Shouguo Yang, Yi Li, Ping Luo, and Yang Liu. 2022. Towards Understanding Third-Party Library Dependency in C/C++ Ecosystem. In *Proc. 37th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*. Article 105.

[16] Yanlin Wang, Hongyu Zhang, Wei Tang, Ping Luo, Shi Han, and Dongmei Zhang. 2023. BBDetector: A Precise and Scalable Third-Party Library Detection in Binary Executables with Fine-Grained Function-Level Features. *Applied Sciences* 13, 1 (2023), 413.

[17] Seunghoon Woo, Sunghan Park, Seulbae Kim, Heejo Lee, and Hakjoo Oh. 2021. CENTRIS: A Precise and Scalable Approach for Identifying Modified Open-Source Software Reuse. In *Proc. 43rd IEEE/ACM Int. Conf. on Software Engineering (ICSE)*. 860–872.

[18] Jiahui Wu, Zhengzi Xu, Wei Tang, Lyuye Zhang, Yueming Wu, Chengyue Liu, Kairan Sun, Lida Zhao, and Yang Liu. 2023. OSSFP: Precise and Scalable C/C++ Third-Party Library Detection using Fingerprinting Functions. In *Proc. 45th IEEE/ACM Int. Conf. on Software Engineering (ICSE)*. 270–282.

[19] Meng Xu, Yongpan Wang, Wei Tang, Xian Zhan, Shing-Chi Cheung, and Hao Yu. 2025. Demystifying Cross-Language C/C++ Binaries: A Robust Software Component Analysis Approach. In *Proc. 40th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*.

[20] Xiaojun Xu, Chang Liu, Qian Feng, Heng Yin, Le Song, and Dawn Song. 2017. Neural Network-Based Graph Embedding for Cross-Platform Binary Code Similarity Detection. In *Proc. 2017 ACM SIGSAC Conf. on Computer and Communications Security (CCS)*. 363–376.

[21] Can Yang, Zhengzi Xu, Hongxu Chen, Yang Liu, Xiaorui Gong, and Baoxu Liu. 2022. ModX: Binary Level Partially Imported Third-Party Library Detection via Program Modularization and Semantic Matching. In *Proc. 44th IEEE/ACM Int. Conf. on Software Engineering (ICSE)*. 1393–1405.

[22] Zimu Yuan, Muyue Feng, Feng Li, Gu Ban, Yang Xiao, Shiyang Wang, Qian Tang, He Su, Chendong Yu, Jiahuan Xu, Aihua Du, Min Yang, and Purui Liu. 2019. B2SFinder: Detecting Open-Source Software Reuse in COTS Software. In *Proc. 34th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*. 1038–1049.

[23] Xian Zhan, Lingling Fan, Sen Chen, Feng Wei, Tianming Liu, Xiapu Luo, and Yang Liu. 2021. ATVHunter: Reliable Version Detection of Third-Party Libraries for Vulnerability Identification in Android Applications. In *Proc. 43rd IEEE/ACM Int. Conf. on Software Engineering (ICSE)*. 1695–1707.

[24] Yayi Zou, Yixiang Zhang, Guanghao Zhao, Yueming Wu, Shuhao Shen, and Cai Fu. 2025. BinCoFer: Three-Stage Purification for Effective C/C++ Binary Third-Party Library Detection. *Journal of Systems and Software* 224 (2025), 112441.